

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Логика и основы алгоритмизации в инженерных задачах»
на тему: «Оценка времени выполнения программ»

Выполнили ст.гр. 25BBB2:

Кашаев Е.А.

Кривошей В.М.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н, ст. преподаватель, Деев М.В.

Пенза 2026

Цель работы: изучить методы оценки времени выполнения программ и сложности алгоритмов, сравнить теоретические оценки с экспериментальными данными.

Ход работы

Дана программа, вычисляющая произведение двух матриц.

Задание 1:

1. Вычислили порядок сложности программы (O-символику).

O-большое — это способ описать скорость роста трудоёмкости алгоритма в зависимости от размера входных данных. Запись $O(\dots)$ отвечает на вопрос: «Как изменится время работы, если данные станут больше?» — и позволяет сравнивать алгоритмы, не привязываясь к конкретному компьютеру и языку программирования.

Например:

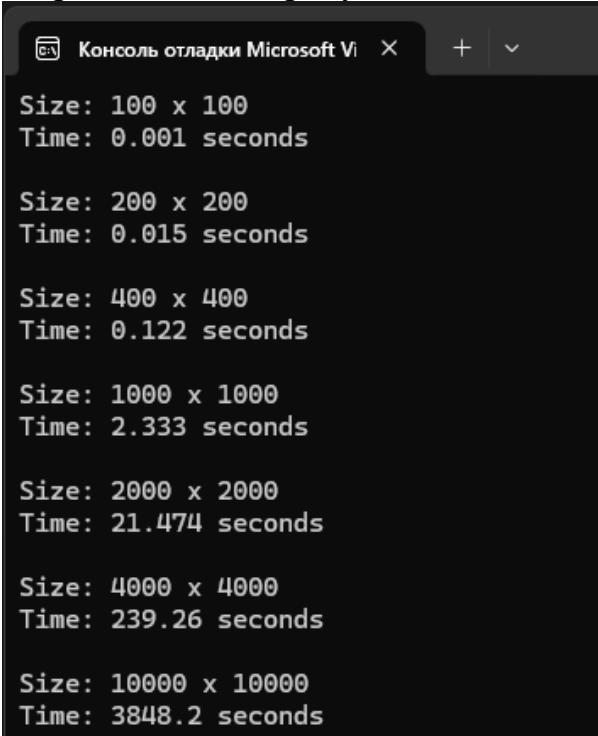
$O(n)$ — время растёт примерно пропорционально n ;

$O(n^2)$ — растёт как квадрат n ;

$O(n^3)$ — растёт как куб n (как и в нашем случае)

2. Оценили время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000

Результат программы 1 представлен на рисунке 1.



```
Size: 100 x 100
Time: 0.001 seconds

Size: 200 x 200
Time: 0.015 seconds

Size: 400 x 400
Time: 0.122 seconds

Size: 1000 x 1000
Time: 2.333 seconds

Size: 2000 x 2000
Time: 21.474 seconds

Size: 4000 x 4000
Time: 239.26 seconds

Size: 10000 x 10000
Time: 3848.2 seconds
```

Рисунок 1

3. Построили график зависимости времени выполнения программы от размера матриц и сравнили полученный результат с теоретической оценкой.

График зависимости времени выполнения программы от размера матриц представлен на рисунке 2.

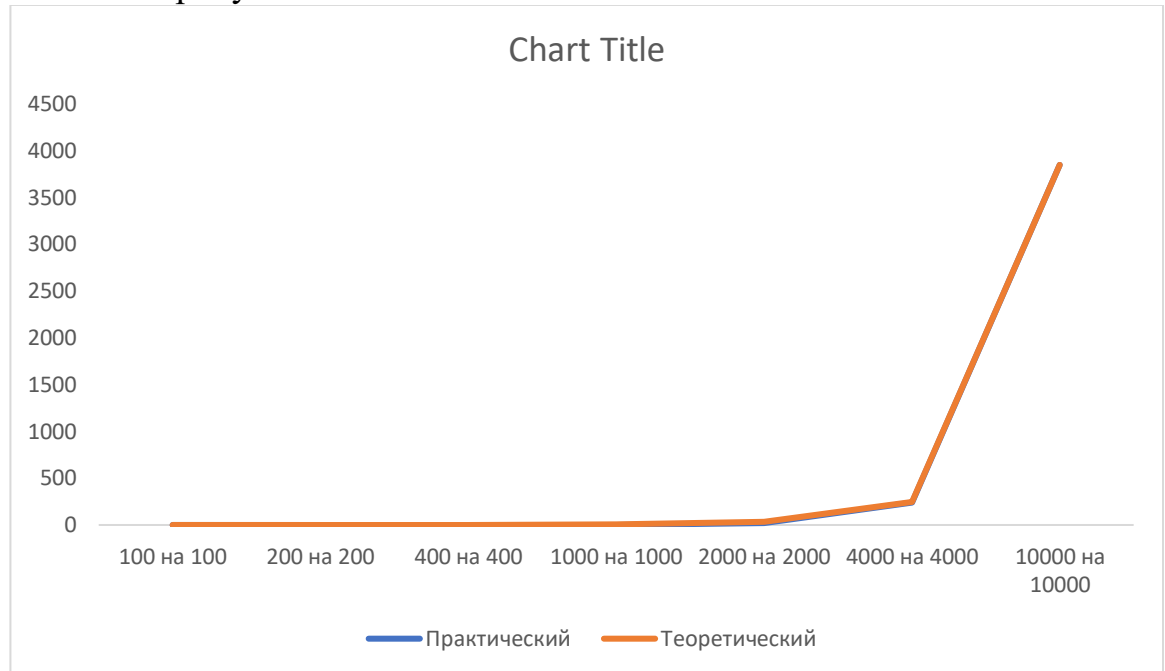


Рисунок 2

На основе полученных данных практический метод измерения времени максимально приблизился к теоретическому методу.

Дана реализация алгоритмов сортировки Шелла и быстрой сортировки

Задание 2:

1. Оценили время работы алгоритма Шелл, быстрой сортировки и стандартной функции `qsort` на массиве 10000.

Результат программы 2 представлен на рисунке 3.

```

Enter array size: 10000

1. Random array:
Shell sort: 0.009000 seconds
Quick sort: 0.001000 seconds
qsort: 0.006000 seconds
Enter array size: 10000

2. Increasing array:
Shell sort: 0.000000 seconds
Quick sort: 0.001000 seconds
qsort: 0.003000 seconds
Enter array size: 10000

3. Decreasing array:
Shell sort: 0.013000 seconds
Quick sort: 0.000000 seconds
qsort: 0.003000 seconds
Enter array size: 10000

4. Half increasing, half decreasing:
Shell sort: 0.004000 seconds
Quick sort: 0.000000 seconds
qsort: 0.004000 seconds

```

Рисунок 3

Алгоритм	Случайный (сек)	Возрастающий(сек)	Убывающий (сек)	Пик (сек)
shell	0,009	0,000	0,013	0,004
qs	0,001	0,001	0,000	0,000
qsort	0,006	0,003	0,003	0,004

В этих тестах **быстрая сортировка (qs)** — **оптимальный выбор**: она стабильно показывает минимальное время на всех типах данных (случайных, отсортированных, обратных, частично упорядоченных) и лучше остальных алгоритмов балансирует скорость и предсказуемость результата.

- Оценили время работы алгоритма Шелл, быстрой сортировки и стандартной функции qsort на массиве 100000.

Результат программы 3 представлен на рисунке 4.

```

Enter array size: 100000

1. Random array:
Shell sort: 0.767000 seconds
Quick sort: 0.017000 seconds
qsort: 0.064000 seconds
Enter array size: 100000

2. Increasing array:
Shell sort: 0.001000 seconds
Quick sort: 0.004000 seconds
qsort: 0.039000 seconds
Enter array size: 100000

3. Decreasing array:
Shell sort: 1.326000 seconds
Quick sort: 0.005000 seconds
qsort: 0.040000 seconds
Enter array size: 100000

4. Half increasing, half decreasing:
Shell sort: 0.321000 seconds
Quick sort: 0.005000 seconds
qsort: 0.038000 seconds

```

Рисунок 4

Алгоритм	Случайный (сек)	Возрастающий(сек)	Убывающий (сек)	Пик (сек)
shell	0,767	0,001	1,326	0,321
qs	0,017	0,004	0,005	0,005
qsort	0,064	0,039	0,040	0,038

Если приоритет — максимальная скорость на разных типах данных, лучше всего подходит qs. Если важнее надёжность и отсутствие резких всплесков времени выполнения — оптимален qsort. Shell имеет смысл применять только в узких случаях, когда данные заведомо почти отсортированы и можно подобрать удачную последовательность шагов; на произвольных данных он проигрывает остальным методам.

Вывод: экспериментально подтверждена кубическая сложность умножения матриц — $O(n^3)$. Сравнение сортировок показало: быстрая сортировка обычно быстрее сортировки Шелла на случайных и убывающих данных, а на возрастающих — наоборот. Стандартная qsort показала стабильное время. Выбор алгоритма зависит от характера входных данных.